Optimizing Breadth-First Search on the NVIDIA DGX Spark

Student: Fabio Missagia
Supervisors: Flavio Vella, Thomas Pasquali, Jacopo Raffi

Department of Information Engineering and Computer Science, University of Trento





NVIDIA DGX Spark: general properties

Property	Value
System	NVIDIA DGX Spark
CPU	20 cores: 10× Cortex-X925 + 10× Cortex-A725 (aarch64)
GPU	NVIDIA GB10 — compute capability 12.1 (`sm_121`)
Memory	128 GB unified LPDDR5x, CPU–GPU coherent, ∼273 GB/s *shared*
L2 cache	24 MiB
SMs	48
GPCs	4 (12 SMs each) — *measured, see next slides*
Max grid	2 147 483 647 × 65 535 × 65 535

One *physical* memory pool, hardware coherent: a kernel dereferences a plain `malloc()` pointer with no `cudaMemcpy`. ∼273 GB/s is an order of magnitude below datacenter HBM, like H100 (∼2.0 TB/s).

NVIDIA DGX Spark: per-SM properties

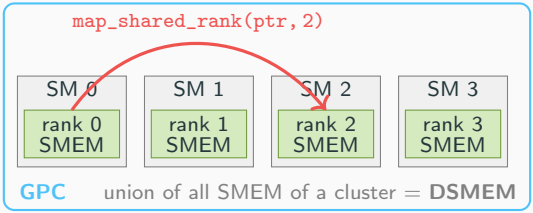
Property	Value
Max threads / block	1024
Max threads / SM	1536 (48 warps)
Max blocks / SM	24
Registers / SM	65 536
Unified L1 + shared memory	128 KiB
Max shared memory / SM	102 400 B (100 KiB)
Default shared memory / block	49 152 B (48 KiB)
Max shared memory / block (opt-in)	101 376 B (99 KiB)
Reserved shared memory / block	1 024 B

L1 and shared memory are the *same* 128 KiB SRAM, split by a configurable carveout: shared memory is bought at the cost of L1.

Thread block clusters (compute capability 9.0+)

An optional level between block and grid:

- **adjacent thread blocks** are grouped into clusters
- guaranteed to be **co-scheduled simultaneously inside one GPC**
- any thread block can address **any peer's shared memory** — loads, stores and *atomics* ride an **SM-to-SM** interconnect, not L2/DRAM ⇒ **DSMEM**
- **co-residency** is guaranteed
- hardware barrier across blocks: `cluster.sync()`



Thread block clusters on the DGX Spark

Measurement	portable	with opt-in
Cluster Size	8	12

- **8** is the maximum CUDA guarantees *portably* on any cluster-capable GPU — code using it runs anywhere.
- **12** is *this hardware's* real maximum, unlocked per kernel with `cudaFuncAttributeNonPortableClusterSizeAllowed` — at the price of code that is no longer portable. Above 12: *cluster misconfiguration*.
- `cudaOccupancyMaxPotentialClusterSize` is **per kernel**, not a device constant: the same kernel answers 8 before the opt-in and 12 after.
- 12 is *usable*, not merely launchable: DSMEM verified across all 12 ranks with a ring test (rank r reads rank $r{+}1 \bmod n$).

How many GPCs?

Problem. No CUDA attribute exposes the GPC count or the SM-to-GPC mapping.

Idea. Exploit the one guarantee available: *all blocks of a cluster are co-scheduled in a single GPC*. So the SMs inside one cluster are provably all in the *same* GPC.

Method. Launch of 4 clusters $\times$ 12 blocks.

- every block reads the `%smid` register and reports the **SM it ran on** and the **cluster it belongs to**
- **52 KB of shared memory per block** forces one block per SM, so a 12-block cluster covers 12 *distinct* SMs
- cluster size 12 = one GPC's SM count, so **4 clusters cover all 48 SMs** exactly once and each cluster is a GPC
- **tried many trials**, but the result is always the same and all 48 SMs are observed every time, so one launch is enough

GPC topology: 4 GPCs × 12 SMs

Example:

GPC	SM ids											
GPC 0	0	1	8	9	16	17	24	25	32	33	40	41
GPC 1	2	3	10	11	18	19	26	27	34	35	42	43
GPC 2	4	5	12	13	20	21	28	29	36	37	44	45
GPC 3	6	7	14	15	22	23	30	31	38	39	46	47

Derived and verified:

- `TPC = smid/2,  GPC = (smid/2) % 4`
- 24 TPCs of 2 SMs, 6 TPCs per GPC, round-robin across GPCs

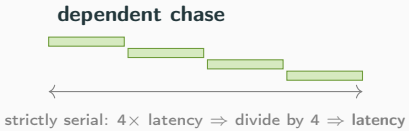
Three independent cross-checks:

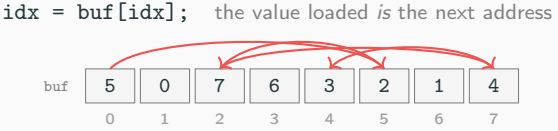
- 4 clusters of 12 *distinct* SMs, 48 SMs observed, none twice
- $4 \times 12 = 48$ matches `multiProcessorCount`
- the largest launchable cluster (12) *equals* one GPC's SM count

How do we measure latency?

A GPU is built to hide latency: a load does not stall the thread, this is called Memory-Level Parallelism.

The fix: make each load depend on the result of the previous one, so that the thread cannot issue the next load until the previous one has completed. This is called a **dependent pointer chase**, or pChase.





In addition:

- **Sattolo cycle:** `buf` is one random cycle over *every* element: no short loops, nothing to prefetch.
- **steps:** how many dependent loads the timed loop performs. If far below the cycle length: no address is revisited.
- **reps:** The whole kernel is launched many times — a drift across reps exposes a cold cache.
- **Warm-up:** L1: one untimed lap to warm the cache; L2: a coalesced sweep *after* the kernel is loaded (lazy load flushes L2).

Latency Benchmark – Random pChase

Dependent random pointer chase, 16 384 steps, one thread, median of 101 reps.

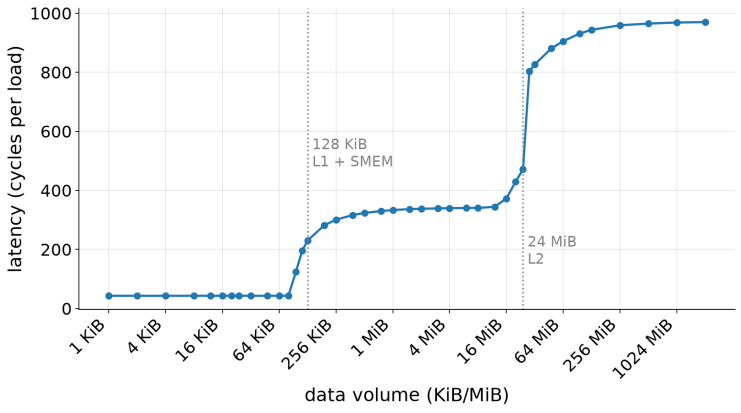
Placement	GB10 (cy)	ns	H800 (cy)*
L1 Cache	43.2	17.9	32.0
SMEM	32.9	13.7	29.0
SMEM, via DSMEM API	36.9	15.3	33.0
DSMEM	180-211	75-88	181-213
L2 Cache	340.1	141.7	264–502
DRAM	965.8	401.6	656

* Luo et al., arXiv:2501.12084 — Hopper H800, same p-chase method

Why does SMEM via DSMEM cost 4 more cycles? `map_shared_rank` returns a generic pointer, so the load compiles to `LD` (address space resolved by hardware on every access) instead of `LDS` (a plain shared-memory load).

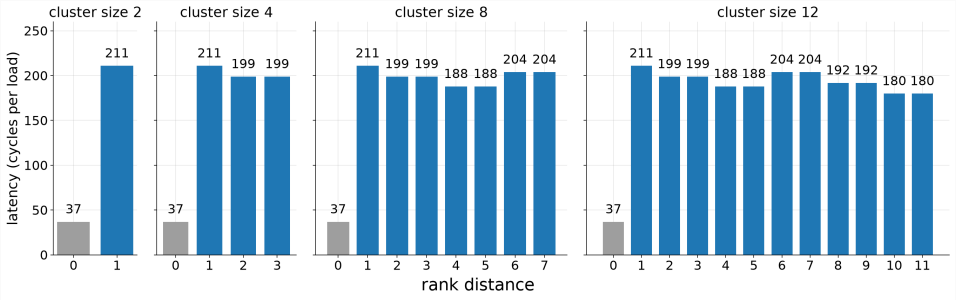
Latency Benchmark – Random pChase

Dependent random pointer chase, 16 384 steps, one thread, median of 101 reps.



Latency Benchmark – Random pChase

Same chase, but thread 0 of rank 0 accesses a *peer* block's DSMEM at rank distance d.



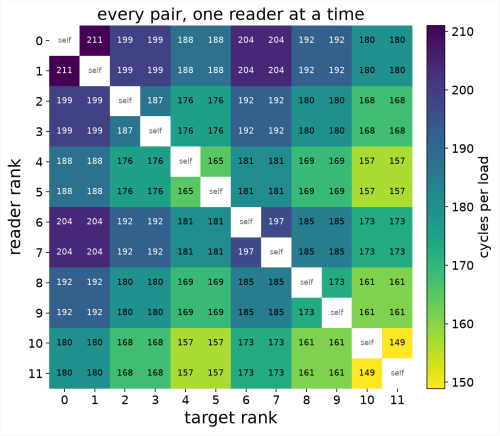
- **A genuine SM-to-SM path.** A remote access costs about *half* an L2 hit and a *fifth* of DRAM.
- **Flat across the cluster.** 210.9 cy at distance 1, identically at cluster sizes 2, 4, 8 and 12. Not monotone in distance: ranks pair up $\{2,3\},\{4,5\},\ldots$ — the two SMs of one TPC.
- **Independently confirmed.** H800 agrees within a few cycles; since GB10's L2 and DRAM are *slower* than Hopper's while its fabric is not, DSMEM's advantage is *larger* here.

Distance does not matter

Rank 0 → its own TPC sibling (rank 1) is 210.9 cy, the *slowest* pair of all...

Rank 0 → the far rank 10 is 179.9 cy, the *fastest* pair of all...

Let's measure every pair!



Facts from the matrix

- perfectly **symmetric**
- TPC siblings have **identical rows**
- the diagonal is **not flat** (211 vs 149)
- **same results** in all 4 GPCs

If distance does not matter, what does?

Over all 36 TPC pairs the following holds with zero error:

$$\text{latency}(k \to d) = 149 + w_k + w_d$$

TPC	5	2	4	1	3	0
w **(cy)**	0	8	12	19	24	31

Maybe a **shared meeting point**?

Every request may go SM → hub → SM.

Independent confirmation: reading *L2* from every SM shows the same pattern!

median L2 latency (cy), per TPC

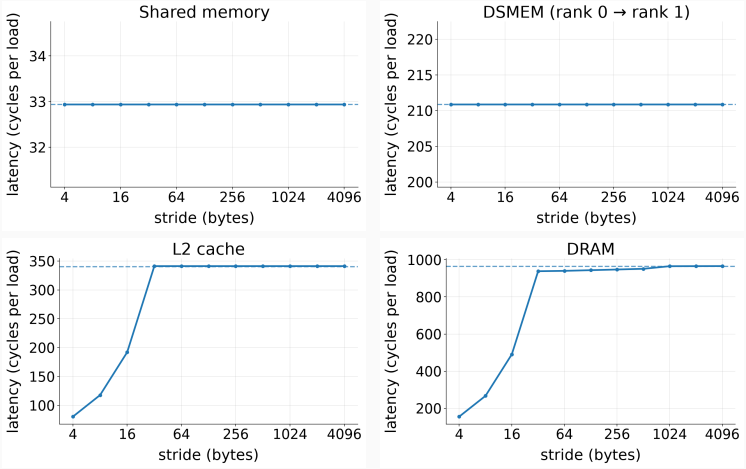
TPC pos.	0	1	2	3	4	5
GPC 0	343	331	320	336	324	312
GPC 1	318	306	295	311	299	287
GPC 2	318	306	295	311	299	287
GPC 3	341	329	318	334	322	310

row minus its minimum = per-TPC distance

TPC pos.	0	1	2	3	4	5
GPC 0	**31**	**19**	**8**	**24**	**12**	**0**
GPC 1	**31**	**19**	**8**	**24**	**12**	**0**
GPC 2	**31**	**19**	**8**	**24**	**12**	**0**
GPC 3	**31**	**19**	**8**	**24**	**12**	**0**

Latency Benchmark – Linear pChase

Same chase, but element i points to $i + s$, $s = 4\,\mathrm{B} \ldots 4\,\mathrm{KiB}$. Dashed = random chase (previous slide).



- **SMEM/DSMEM are pattern-blind**
- **L2/DRAM see only the 32 B sector** $k = 32/s$ loads share one L1 fill, $(T + (k-1) \cdot 43.2)/k$ gives 80/118/192 cy (L2) and 155/266/490 cy (DRAM) — as measured.
- **No hardware prefetcher:** from 32 B on, linear = random.

Clusters are not free

The moment a launch carries a cluster dimension, the device holds **2 blocks per SM at most (96 in total), whatever the block size.**

blockDim	total blocks (unclustered)	total blocks (clustered)	penalty	occupancy (clustered)
64	1152	96	12×	8%
128	576	96	6×	17%
256	288	96	3×	33%
512	144	96	1.5×	67%
1024	48	48	none	67%

- **Confirmed two independent ways:** `cudaOccupancyMaxActiveClusters`, and a test in which every block must observe all the others before a timeout.
- **Didn't found anything about this on the web...**

Clusters are not free

The cap also moves with the *shared memory* a block requests. Past ∼50 KiB only one block fits per SM, so the device holds 48.

SMEM / block	cs = 1	cs = 2	cs = 4	cs = 8	cs = 12
0	96	96	96	96	96
32 KiB	96	96	96	96	96
50 KiB	48	48	48	**32**	48
99 KiB	48	48	48	**32**	48

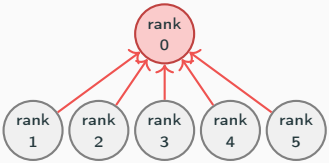
total resident blocks device-wide (48 SMs), cs = cluster size

- A cluster never spans a GPC, and a GPC has 12 SMs. One cluster of 8 leaves 4 SMs free — too few for a second — so each of the 4 GPCs runs one cluster and strands 4 SMs: $4 \times 8 = 32$, a third of the GPU idle.
- **12 and 6 tile the machine exactly**; 8, the *portable* maximum, does not.

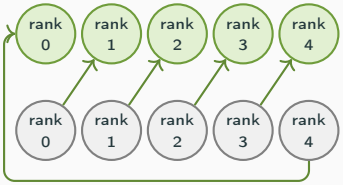
Throughput Benchmark

So far: *one* thread reading *one* peer. Now: *many* threads, reading the *same* peer or *different* peers.

all vs 1 (`broadcast`)



all vs all (`ring`)

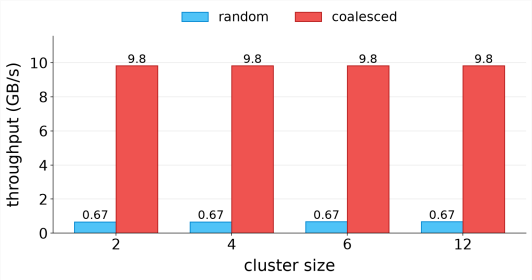


- Two access patterns:
  - `random`: each lane starts from a different random position in the same cycle, resulting in approximately 32 load chains per warp.
  - `coalesced`: lane l starts at $base + l \Rightarrow$ a warp reads 32 consecutive words, one 128 B line, a *single* request.

Throughput Benchmark – Broadcast

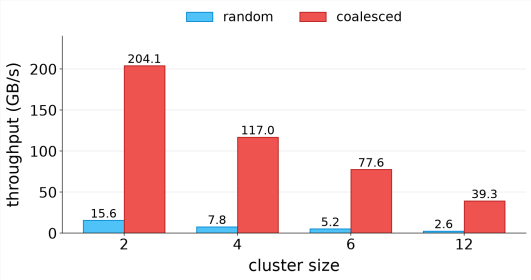
In every cluster, every thread of ranks $1 \ldots N-1$ reads **rank 0**'s shared memory.
1 block per SM, 256 threads per block.

One cluster



1 cluster of N blocks: only N SMs busy.

Whole GPU



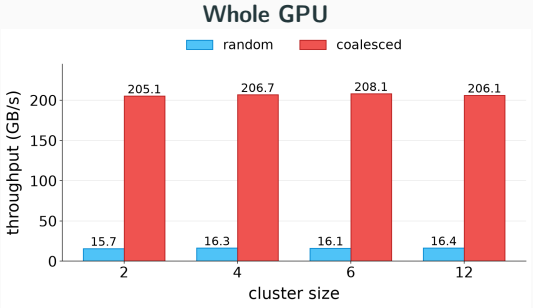
$48/N$ clusters of N blocks side by side: all 48 SMs busy.

Throughput Benchmark – Ring

In every cluster, every thread of rank k reads **rank** $k+1 \pmod N$: one reader per owner.
1 block per SM, 256 threads per block.



1 cluster of N blocks: only N SMs busy.



48/N clusters of N blocks side by side: all 48 SMs busy.

Throughput Benchmark – Summary

scope	**coalesced**	random	what sets the limit
one owner SM	**9.8 GB/s**	0.67 GB/s	the owner: 128 B every ∼31 cy ≈ 4.1 B/cy
one GPC (12 SMs)	∼**53 GB/s**	4.3 GB/s	the GPC's SM-to-SM network
whole chip (48 SMs)	∼**206 GB/s**	16 GB/s	4 GPCs, independent

- **coalesced** is limited by *bytes*, **random** by *requests* (one every ∼14 cy per owner, only 4 B each): 15× apart.
- The owner here is SM 0, the worst position in the GPC; the best-placed owner reaches ∼17 GB/s (∼7 B/cy).
- The GPC network saturates with ∼6 SMs reading: beyond that, adding SMs only splits the same ∼53 GB/s.

What's next?

1. **Characterization of DSMEM**
   - Performance characterization and impact of DSMEM
   - Limitations and assumptions of the DSMEM programming paradigm
   - Theoretical upper bounds on performance improvements
   - Applicability: algorithms that can and cannot exploit DSMEM
2. **State-of-the-art GPU-accelerated BFS**
   - Microbenchmark access pattern of a BFS?
   - Graph partitioning strategies
   - Work-stealing techniques
   - Data types and data structures for high-performance BFS
3. **BFS implementation based on DSMEM**
   - Algorithm design informed by (1) and (2)
   - Graph partitioning (probably 2D) and work stealing
   - Integration of *mxint* (micro-scaling integer type), if worthwhile
   - Performance evaluation; limitations and future directions

Barriers: how we measure them naively

```
__global__ void barrier() {
  long long start = clock64();
  barrier.sync();
  long long end = clock64();
  arrive_time[blockIdx.x][threadIdx.x]
      = end - start;
}
```

- **every thread** times its own wait at the barrier
- one sample per thread ⇒ a distribution, not one number
- `barrier` is one of four scopes:
  - `__syncthreads()`: one block
  - `cluster.sync()`: one cluster
  - cluster barrier with a *relaxed* arrive
  - `grid.sync()`: the whole grid
- 32–1024 threads per block; 2 and 12 blocks

PTX & SASS of `__syncthreads()`

PTX

```
mov.u64  %rd1, %clock64;
bar.sync 0;
mov.u64  %rd2, %clock64;
```

SASS

```
CS2UR UR6, SR_CLOCKLO
BAR.SYNC.DEFER_BLOCKING 0x0
CS2R  R2, SR_CLOCKLO
```

- one PTX instruction → **one** SASS instruction: a hardware barrier inside the SM, no memory fence, no traffic outside the SM
- `DEFER_BLOCKING`: the warp is not stopped *at* the barrier but at the next instruction that depends on it
- the clock read (`CS2R`) does not touch memory ⇒ it can run **before** the barrier is over

PTX & SASS of `cluster.sync()`

PTX

```
barrier.cluster.arrive;   // release
barrier.cluster.wait;     // acquire
```

- **two paths**, chosen at run time by `%is_explicit_cluster`: cluster barrier (`UCGABAR_ARV/WAIT`), or `__syncthreads()` when not launched with clusters
- the release is a **GPU-scope** fence (`MEMBAR.ALL.GPU`), although only the cluster has to see the writes
- the acquire invalidates the whole L1 (`CCTL.IVALL`)

SASS

```
  LDCU UR8, c[0x0][0x36c]    // %is_explicit_cluster
  UISETP.EQ.U32.AND UP0, UPT, UR8, 0x1, UPT
  BRA.U !UP0, .L_x_0
  MEMBAR.ALL.GPU             // release
  ERRBAR
  CGAERRBAR
  UCGABAR_ARV                // arrive
  BRA .L_x_1
.L_x_0:
  NOP
.L_x_1:
  BRA.U !UP0, .L_x_2
  UCGABAR_WAIT               // wait
  CCTL.IVALL                 // acquire
  BRA .L_x_3
.L_x_2:
  BAR.SYNC.DEFER_BLOCKING 0x0  // no cluster
.L_x_3:
```

What is a fence?

A **fence** keeps a thread's memory accesses in order: no thread in the fence's **scope** (block, cluster, GPU) can see what the thread did after the fence without also seeing what it did before. A barrier that publishes data needs two halves:

- **release** (on arrive): my earlier writes are visible to whoever passes the barrier
- **acquire** (on wait): my later reads see the others' writes, not stale copies

PTX fences compiled for `sm_121`:

PTX	SASS: release half			acquire half
`fence.acq_rel.cta`	`MEMBAR.ALL.CTA`			
`fence.acq_rel.cluster`	`MEMBAR.ALL.GPU`	`ERRBAR`	`CGAERRBAR`	`CCTL.IVALL`
`fence.acq_rel.gpu`	`MEMBAR.ALL.GPU`	`ERRBAR`	`CGAERRBAR`	`CCTL.IVALL`
`fence.sc.gpu`	`MEMBAR.SC.GPU`	`ERRBAR`	`CGAERRBAR`	`CCTL.IVALL`
`fence.acq_rel.sys`	`MEMBAR.ALL.SYS`	`ERRBAR`	`CGAERRBAR`	`CCTL.IVALL`
`barrier.cluster.arrive`	`MEMBAR.ALL.GPU`	`ERRBAR`	`CGAERRBAR`	
`barrier.cluster.wait`				`CCTL.IVALL`

No cluster scope in hardware!

Reading the `cluster.sync()` SASS

```
MEMBAR.ALL.GPU
ERRBAR
CGAERRBAR
UCGABAR_ARV
...
UCGABAR_WAIT
CCTL.IVALL
```

- `MEMBAR.ALL.GPU`: memory barrier; waits until the warp's earlier loads and stores are performed at the scope (`CTA`, `GPU`, `SYS`). `ALL` = acq_rel, `SC` = sequentially consistent
- `ERRBAR`: error barrier; waits until earlier memory operations can no longer report a fault
- `CGAERRBAR`: the same for cluster (DSMEM) traffic. CGA, *Cooperative Grid Array*, is NVIDIA's hardware name for a cluster
- `UCGABAR_ARV`, `UCGABAR_WAIT`: the hardware cluster barrier, arrive and wait. `U` = uniform datapath: executed once per warp
- `CCTL.IVALL`: cache control, invalidate all of L1, so later loads go to L2 and see the other SMs' writes

Only the names are documented (CUDA Binary Utilities, instruction set reference); the semantics are our reading of them and of the PTX they come from.

PTX & SASS of the cluster barrier, relaxed arrive

PTX

```
barrier.cluster.arrive.relaxed.aligned;
barrier.cluster.wait.aligned;
```

- `.relaxed`: the arrive makes **no** write visible
- same SASS as `cluster.sync()` **minus the release**: the three fences before `UCGABAR_ARV` are gone
- the wait keeps its acquire (`CCTL.IVALL`)
- relaxed vs. default = the cost of the release fences

SASS

```
  LDCU UR8, c[0x0][0x36c]    // %is_explicit_cluster
  UISETP.EQ.U32.AND UP0, UPT, UR8, 0x1, UPT
  BRA.U !UP0, .L_x_0
                             // (no release fences)
  UCGABAR_ARV                // arrive
  BRA .L_x_1
.L_x_0:
  NOP
.L_x_1:
  BRA.U !UP0, .L_x_2
  UCGABAR_WAIT               // wait
  CCTL.IVALL                 // acquire
  BRA .L_x_3
.L_x_2:
  BAR.SYNC.DEFER_BLOCKING 0x0  // no cluster
.L_x_3:
```

Taking `cluster.sync()` apart: what each piece costs

Loop of 1000 barriers (after 100 untimed), same results for cluster sizes 1, ..., 12.

piece	32 thr	1024 thr
full `cluster.sync()`	377 cy	617 cy
relaxed arrive (no fence)	96 cy	145 cy
↪ arrive alone	44 cy	62 cy
↪ wait, all arrived	24 cy	28 cy

PTX	SASS	32 thr	1024 thr
`fence.acquire.cluster`	`CCTL.IVALL`	4 cy	98 cy
`fence.release.cluster`	`MEMBAR.ALL.GPU`	281 cy	312 cy
`fence.acq_rel.cluster`	both halves	282 cy	326 cy
`fence.acq_rel.cta`	`MEMBAR.ALL.CTA`	7 cy	59 cy

`cluster.sync()` with 1 block of 32 threads ≈ 380 cycles



- **the release fence is ∼75% at every block size**: 281 of 377 cycles at 32 threads, 472 of 617 at 1024
- **above 8 warps the fences queue**: released together, the slowest waits +75 (512 thr), +132 (1024 thr)
- **cheap fences grow with the warps**: the acquire is free only for small blocks (4 → 98 cycles)
- **same cost for 1 block and for 12**: a fixed round trip to a unit, not messages between blocks
- **the 1st barrier of cluster.sync() costs ∼90 more**
- a wait on a completed barrier costs 24 ⇒ **split barrier**: arrive early, wait late

Cost by TPC position, for each GPC

48 clusters of one block, one per SM, all at once; 32 threads, loop of 1000. Values in cycles.

		same in all 4 GPCs			**L2 load, depends on the GPC**			
TPC pos.	w	`cluster.sync()`	relaxed	fence	GPC 0	GPC 1	GPC 2	GPC 3
0	31	377	96	282	343	318	318	341
3	24	364	89	276	336	311	311	334
1	19	353	84	270	331	306	306	329
4	12	340	77	264	324	299	299	322
2	8	333	77	262	320	295	295	318
5	0	326	77	262	312	287	287	310

- rows go from the farthest TPC ($w = 31$) to the closest ($w = 0$)
- **barrier and fence do not depend on the GPC**; L2 does: GPCs 0 and 3 cost ∼25 more

PTX & SASS of `grid.sync()`

PTX (abridged)

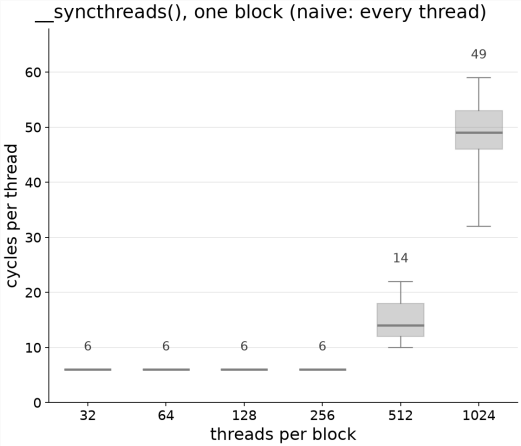
```
  mov.u32 %r3, %envreg2;       // grid workspace
  mov.u32 %r4, %envreg1;       // (set by the driver)
  setp.ne.s64 %p1, %rd1, 0;
  @%p1 bra $L__BB3_2;
  trap;                        // not cooperative
$L__BB3_2:
  barrier.sync 0;              // (1) block barrier
  setp.ne.s32 %p2, %r8, 0;
  @%p2 bra $L__BB3_5;          // thread 0 only
  ...
  selp.b32 %r10, %r21, 1, %p3;
  atom.add.release.gpu.u32 %r9, [%rd9], %r10;
$L__BB3_4:                     // (2) spin
  ld.acquire.gpu.u32 %r22, [%rd9];
  xor.b32 %r23, %r22, %r9;
  setp.gt.s32 %p4, %r23, -1;  // top bit flipped?
  @%p4 bra $L__BB3_4;
$L__BB3_5:
  barrier.sync 0;              // (3) block barrier
```

No hardware grid barrier: a counter in global memory (L2). Block 0 adds $0x80000001-n$, the others add 1: the n adds sum to 0x80000000, so the top bit flips when the last block arrives.

SASS (abridged)

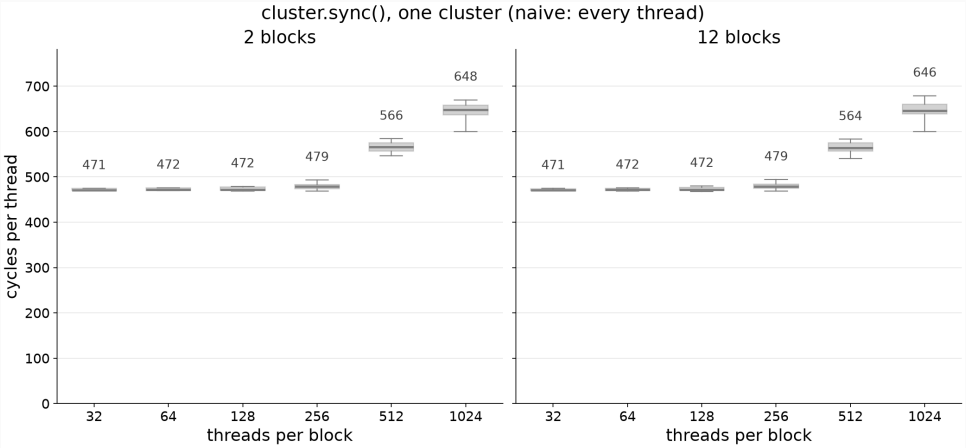
```
  ISETP.NE.S64.AND P0, PT, R2, RZ, PT
  @P0 BRA .L_x_0
  BPT.TRAP 0x1                 // not cooperative
.L_x_0:
  BAR.SYNC.DEFER_BLOCKING 0x0 // (1)
  LOP3.LUT P0, RZ, R0, R5, RZ, 0xfc, !PT
  @P0 BRA .L_x_1               // thread 0 only
  VOTEU.ANY UR14, UPT, PT      // warp-aggregated
  FLO.U32 R4, UR14             // atomic: a leader
  POPC R0, UR14                // lane adds for all
  ...
  @P0 MEMBAR.ALL.GPU           // release
  @P0 ERRBAR
  @P0 CGAERRBAR
  @P0 ATOM.E.ADD.STRONG.GPU PT, R5, [R2.64+0x4], R5
  SHFL.IDX PT, R0, R5, R4, 0x1f
.L_x_2:                        // (2) spin
  LD.E.STRONG.GPU R5, [R2.64+0x4]
  CCTL.IVALL                   // acquire
  YIELD
  LOP3.LUT R5, R5, R0, RZ, 0x3c, !PT
  ISETP.GT.AND P0, PT, R5, -0x1, PT
  @P0 BRA .L_x_2
.L_x_1:
  WARPSYNC.ALL
  BAR.SYNC.DEFER_BLOCKING 0x0 // (3)
```

Naive: `__syncthreads()`



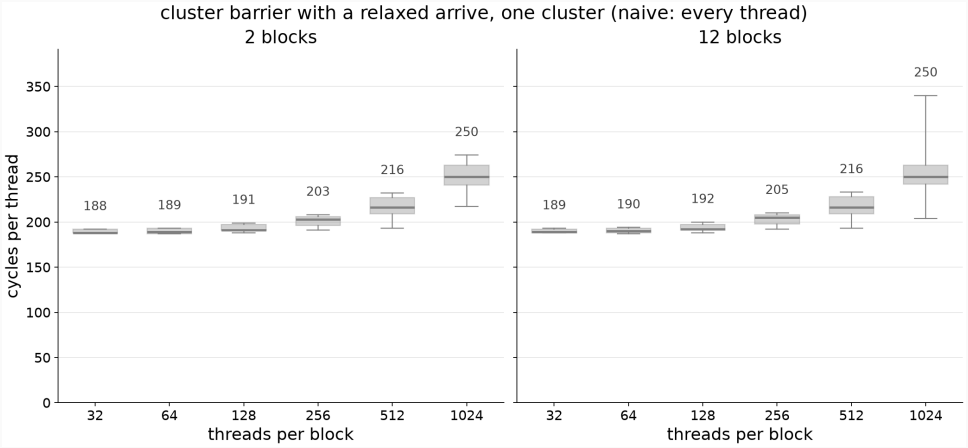
6 cycles up to 256 threads, then 14 (512) and 49 (1024).

Naive: `cluster.sync()`



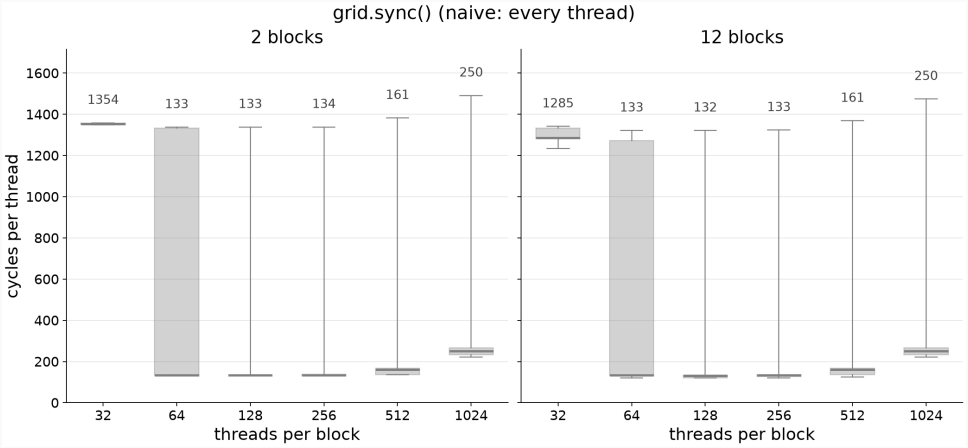
~472 cycles up to 256 threads; the same with 2 and 12 blocks.

Naive: cluster barrier, relaxed arrive



~190 cycles: less than half of `cluster.sync()`.

Naive: `grid.sync()`



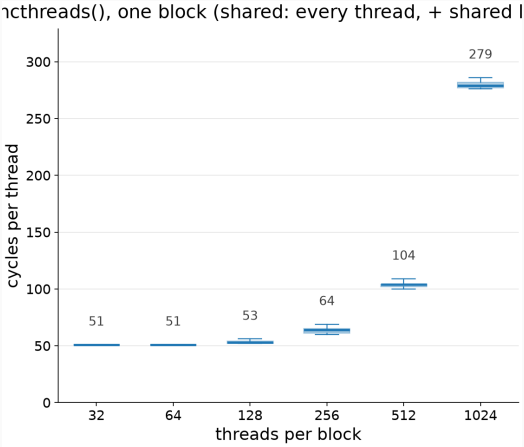
Median ∼133 cycles, *less* than `cluster.sync()`, but whiskers up to ∼1300: suspicious.

Barriers: how we measure them avoiding optimizations

```
__global__ void barrier() {
  __shared__ unsigned word;
  long long start = clock64();
  barrier.sync();
  // read from word from shared memory;
  long long end = clock64();
  arrive_time[blockIdx.x][threadIdx.x]
      = end - start;
}
```

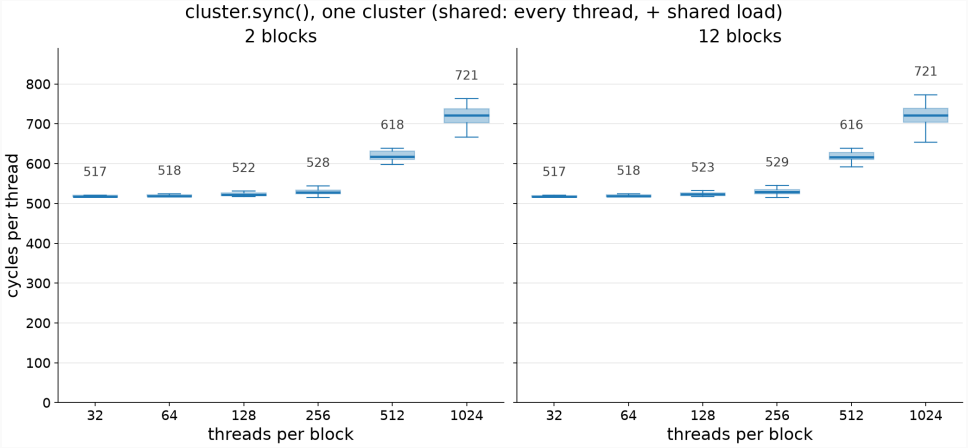
- **every thread** times its own wait at the barrier
- one sample per thread ⇒ a distribution, not one number
- `barrier` is one of four scopes:
  - `__syncthreads()`: one block
  - `cluster.sync()`: one cluster
  - cluster barrier with a *relaxed* arrive
  - `grid.sync()`: the whole grid
- 32–1024 threads per block; 2 and 12 blocks

With shared load: `__syncthreads()`



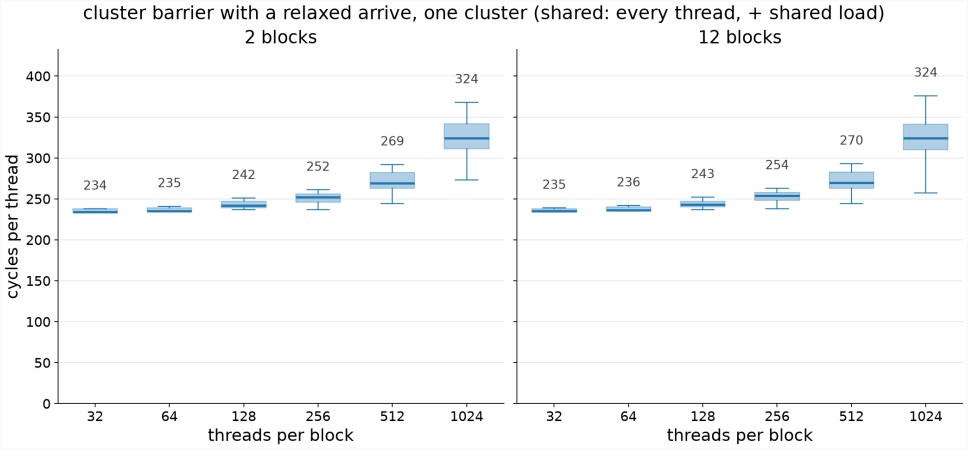
51 cycles at 32 threads: the load itself, the barrier is free.

With shared load: `cluster.sync()`



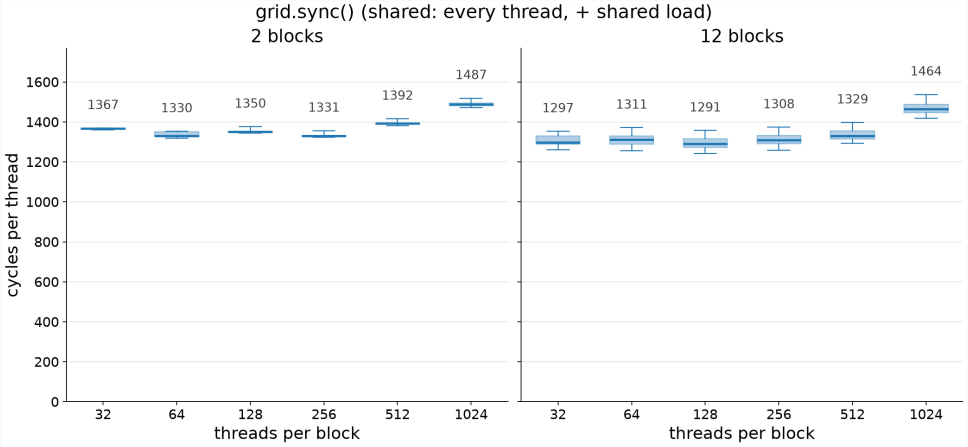
$\sim$517 cycles: naive value + the load ($\sim$50). The naive number was right.

With shared load: cluster barrier, relaxed arrive



$\sim$235 cycles: again naive + the load.

With shared load: `grid.sync()`



~1300 cycles for *every* thread, not 133: the naive `grid.sync()` was wrong.

A closer look: only warp 0 is right

Looking at the naive `grid.sync()` warp by warp:

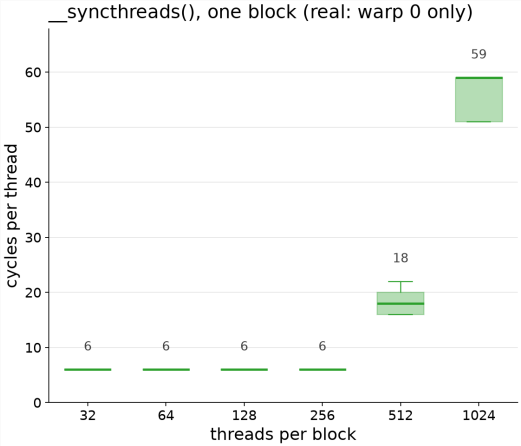
- **warp 0** always measures ∼1270–1350 cycles;
- **all the other warps** measure ∼133 cycles, although they wait just as long.

Probably the barrier is **not strictly blocking** when it does not have to be: it compiles to `BAR.SYNC.DEFER_BLOCKING`, so a warp is stopped a few instructions *after* the barrier, and a `clock64()` read (not a memory access) can slip in before the wait is over.

Warp 0 does the work of `grid.sync()` (the atomic in global memory and the spin on it), so its clock read comes after the barrier is really over.

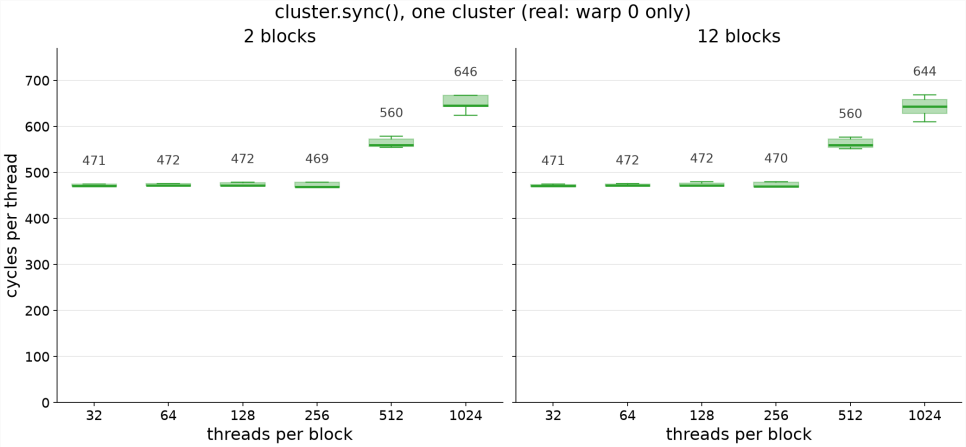
⇒ measure the naive way, but keep **only warp 0 of each block**: no shared load, so no extra ∼50 cycles.

Warp 0 only: `__syncthreads()`



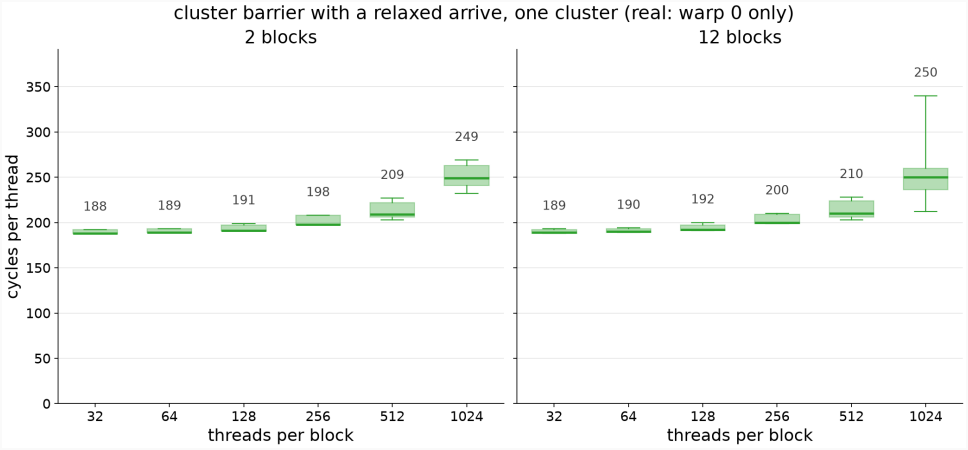
6 cycles up to 256 threads, 18 (512), 59 (1024): same picture as the naive one.

Warp 0 only: `cluster.sync()`



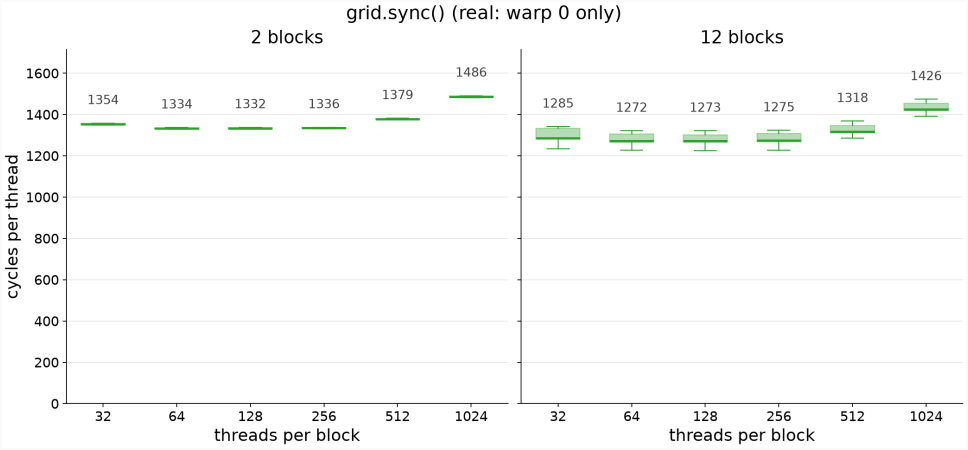
$\sim$471 cycles: identical to the naive median.

Warp 0 only: cluster barrier, relaxed arrive



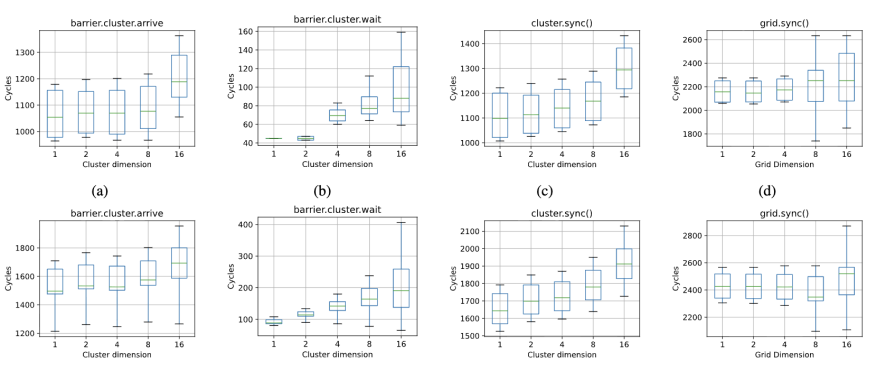
$\sim$190 cycles: identical to the naive median.

Warp 0 only: `grid.sync()`



$\sim$1270–1350 cycles up to 256 threads: the real cost, without the $\sim$50 cycles of the shared load.

Comparison: *Benchmarking Thread Block Clusters*



The authors measured on an **H100 PCIe** (114 SMs); we measure on GB10 (48 SMs). `cluster.sync()`: ∼1100–1300 cycles on H100 vs ∼470 here; `grid.sync()`: ∼2150–2250 vs ∼1250–1320 here ⇒ roughly **half the cycles** on the smaller GPU.

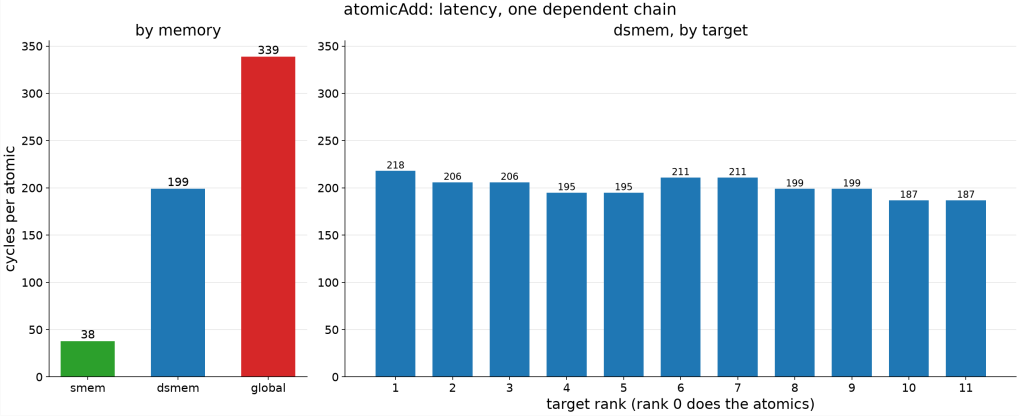
Atomics: latency

`atomicAdd` vs `atomicOr` vs `atomicCAS`

```
__shared__ unsigned buf[SBUF];
cg::cluster_group cluster = cg::this_cluster();
if (cluster.block_rank() == 0 && threadIdx.x == 0) {
  // MEMORY: own_smem | remote_smem | global_mem
  unsigned* c = MEMORY(buf, global_buf, cluster, target_rank);
  unsigned i = 0;   // word 0 holds 0, and every atomic leaves it 0
  long long start = clock64();
  for (int s = 0; s < steps; s++) {
    i = CHAIN(&c[i]);   // chain_or | chain_add | chain_cas
  }
  long long cycles = clock64() - start;
}
cluster.sync();
```

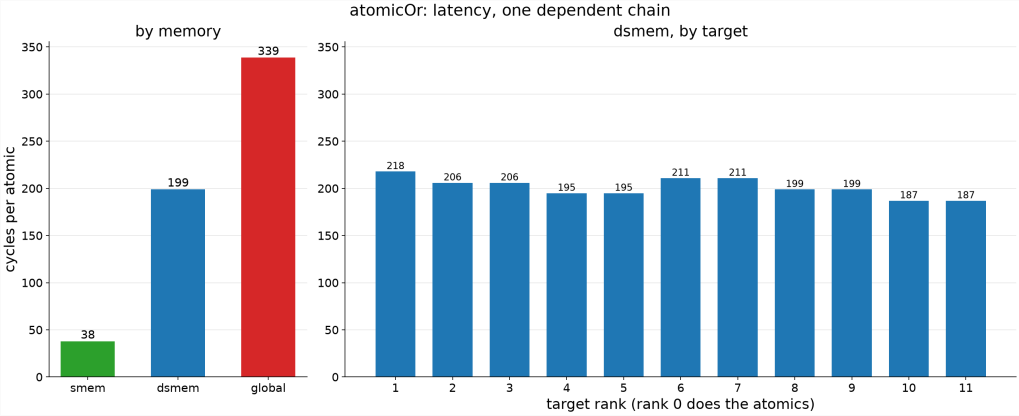
- **one thread**, a chain of dependent atomics: each works on the word the previous one returned ⇒ pure latency
- own **smem**, a peer's **DSMEM**, **global** (L2)
- DSMEM: rank 0 against every target rank 1–11

Latency: `atomicAdd`



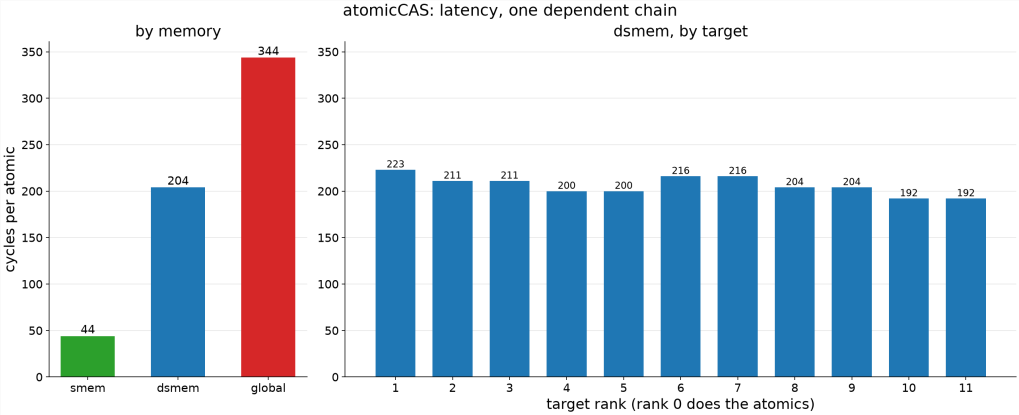
smem 38, DSMEM 199, global 339 cycles. DSMEM depends on the target TPC, like a plain load (187–218).

Latency: `atomicOr`



Identical to `atomicAdd`: 38 / 199 / 339 cycles.

Latency: atomicCAS



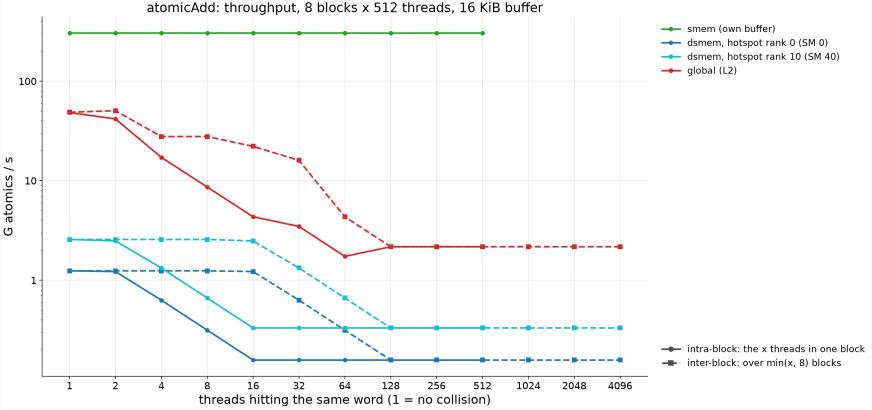
About 5 cycles more everywhere: 44 / 204 / 344 cycles.

Atomics: throughput under contention

One cluster of 12 blocks. **8 worker blocks** $\times$ **512 threads** $= 4096$ threads, each doing many atomics on a 16 KiB buffer.

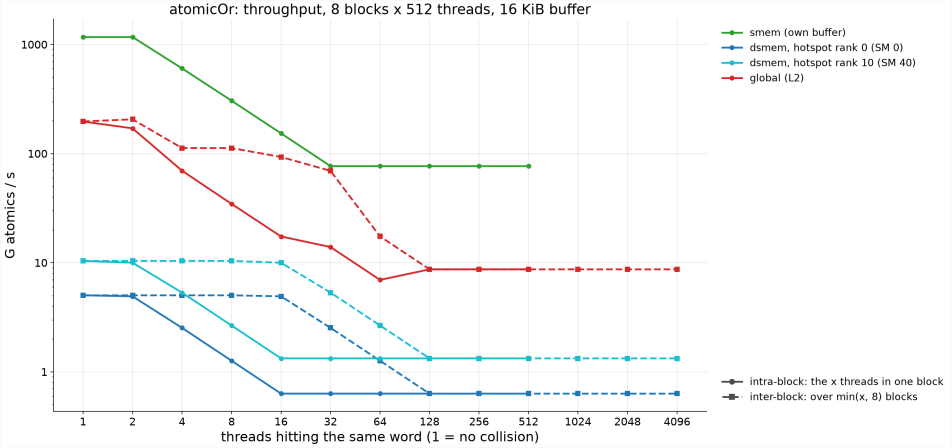
- **Contention** x: x threads share each word ($x = 1$: no collision, $x = 4096$: everybody on one word).
- **Who collides:**
  - *Intra-block* (solid): the x threads that share a word are neighbours in one block, lanes of the same warp up to $x = 32$. Example ($x = 4$): threads 0–3 of rank 1 $\rightarrow$ word 0; threads 0–3 of rank 2 $\rightarrow$ word 128.
  - *Inter-block* (dashed): the x threads that share a word are spread over $\min(x, 8)$ blocks, i.e. over different SMs. Example ($x = 4$): thread t of ranks 1–4 $\rightarrow$ word t.
- **Where the buffer is:** each worker's own smem; one block's DSMEM that every worker targets (the *hotspot*: rank 0 on SM 0, or rank 10 on SM 40); global memory.
- **Time:** from when the first worker starts to when the last one finishes.
- Result: atomics per second, all workers together. Every run is checked word by word.

Throughput: `atomicAdd`



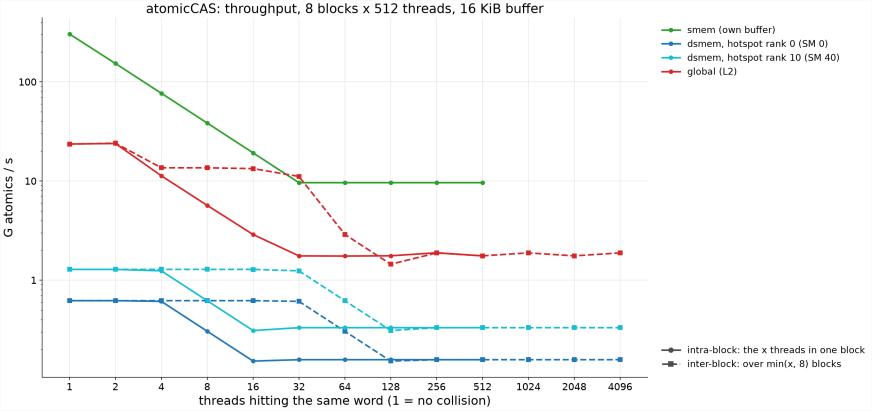
smem stays flat ($\sim$300 G/s): the hardware merges a warp's adds on one word. Global 48 G/s, DSMEM 1.2–2.5 G/s at $x = 1$.

Throughput: `atomicOr`



Same shape, $\sim 4\times$ higher: smem 1150, global 200, DSMEM 5–10 G/s at $x = 1$.

Throughput: `atomicCAS`



The slowest: smem 300, global 24, DSMEM 0.6–1.3 G/s at $x = 1$, and smem also drops with contention.

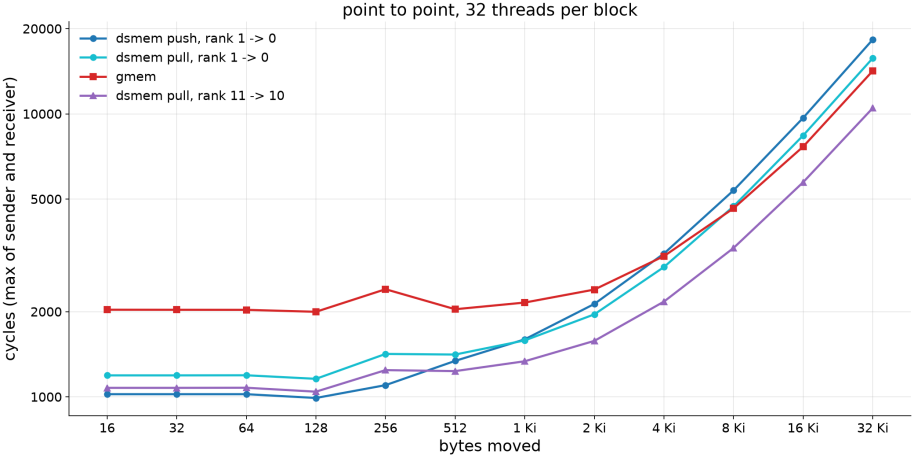
Communication between SMs: DSMEM vs GMEM

The simplest case: **two SMs exchange x bytes**. Three ways to do it:

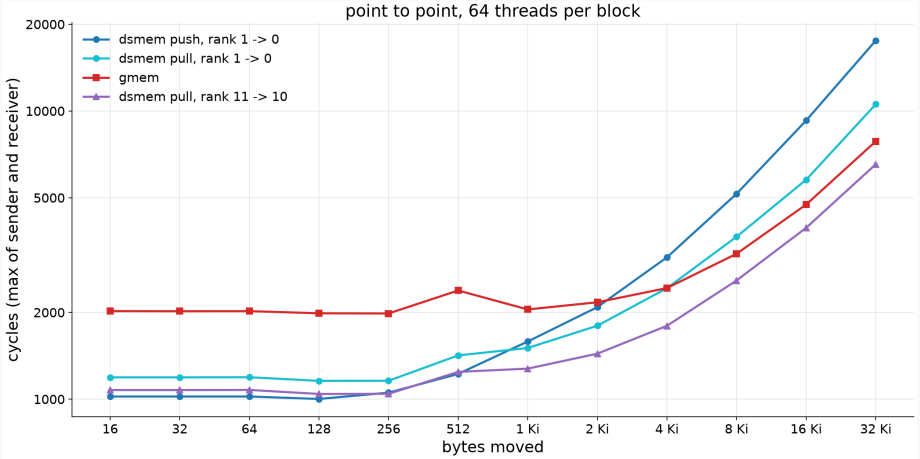
- **DSMEM push**: block A writes directly into block B's shared memory, then B reads it.
- **DSMEM pull**: block A writes into its *own* shared memory, block B reads it from there.
- **GMEM**: block A writes into a global-memory buffer; after synchronization, block B reads it.

Time = cycles of the slower of sender and receiver, synchronization included; x from 16 B to 32 KiB, 32–1024 threads per block.

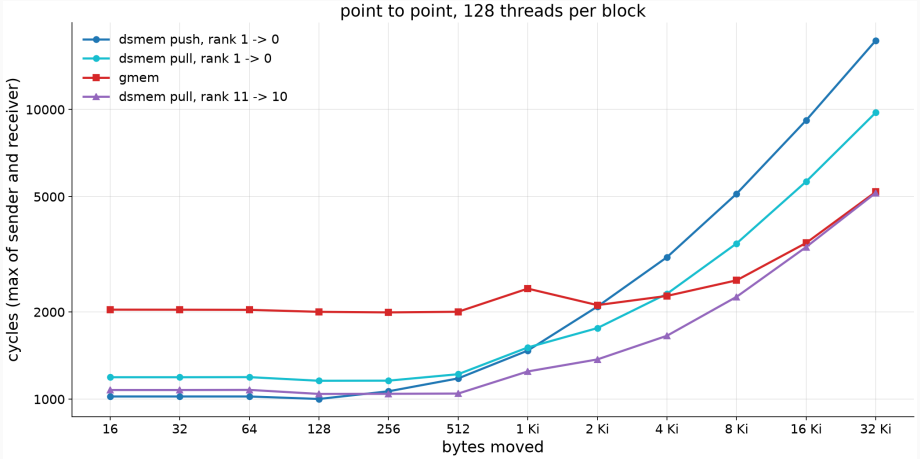
Point to point: 32 threads per block



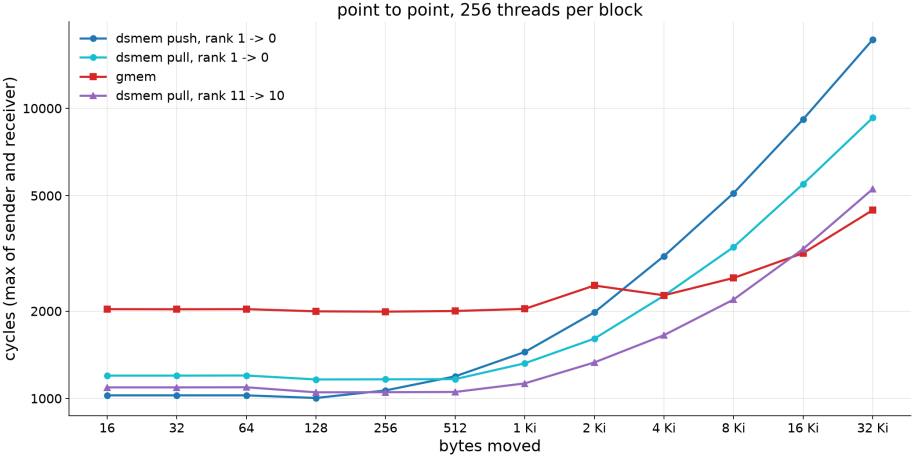
Point to point: 64 threads per block



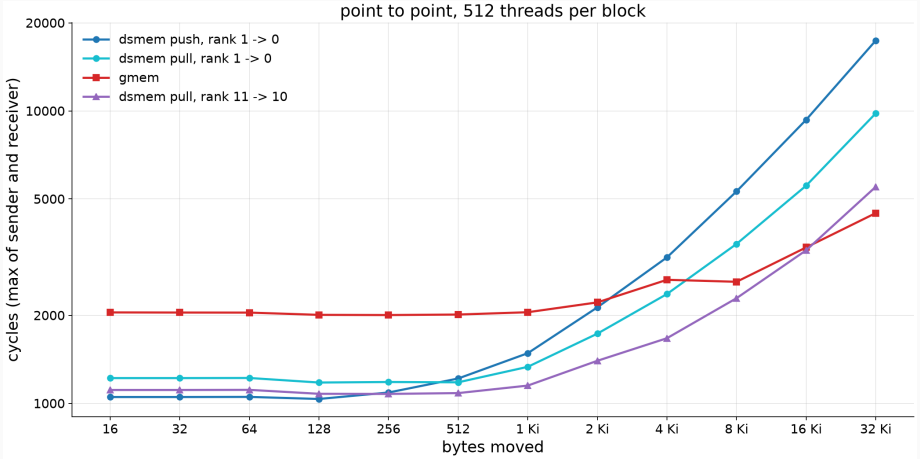
Point to point: 128 threads per block



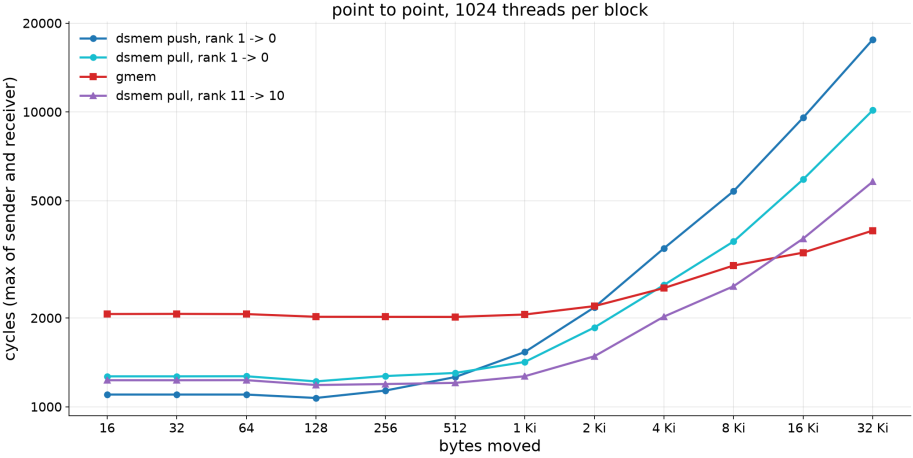
Point to point: 256 threads per block



Point to point: 512 threads per block



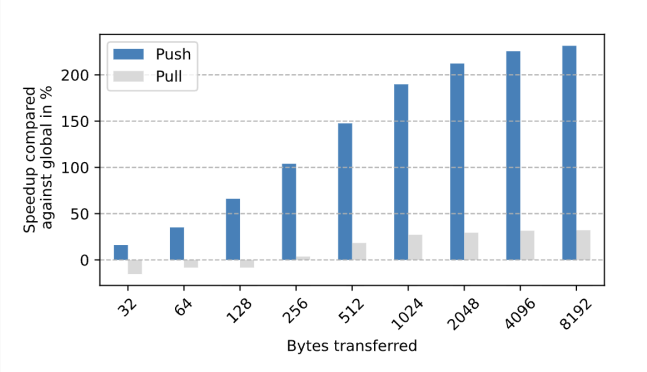
Point to point: 1024 threads per block



Point to point: what we learn

- **DSMEM outperforms GMEM for exchanges $\leq$ 4 KiB**: no round trip through L2.
- Above that, GMEM wins: it moves large buffers faster than the SM-to-SM path.
- **It depends a lot on which SMs talk**: pull from rank 11 $\rightarrow$ 10 is clearly faster than rank 1 $\rightarrow$ 0 — again the interconnect topology (TPC distance to the hub).

Comparison: *Benchmarking Thread Block Clusters*



The same paper reports DSMEM's speedup over global memory, but using H100 PCIe with **32 threads** moving `uint8_t` (**1 B**) elements...

Unbalanced communication: all-to-all-v in a cluster

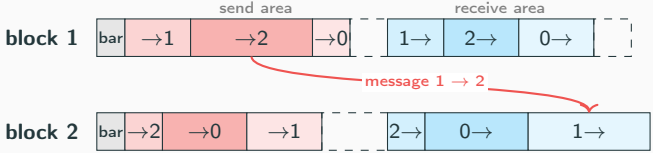
A 2D-partitioned BFS, at every level, hands each discovered vertex to the block that owns it: every block sends a **different** number of bytes to every other block (all-to-all-*v*), and on power-law graphs the amounts are **skewed**.

- **Where the sizes come from:** `axccl` generates unbalanced patterns for MPI collectives (HEAVYTAIL, UNIMODALFLAT, ...) and writes them to a trace, one line per (source, destination) pair.
- `throughput.cu` **replays** the trace: MPI rank $i \to$ block i of one cluster (up to 12), and $\mathit{sendsize}(i \to j) \to$ bytes block i delivers into block j's shared memory. No MPI: the file is only a recipe.
- **One exchange:** `cluster.sync()` $\to$ move the bytes $\to$ `cluster.sync()`. Time per exchange averaged over many rounds; the two barriers alone are the floor.
- Every block also records its **busy time**: heavy senders arrive last, the others wait.
- Correctness first: every received word is checked against the pattern.

Unbalanced communication: each block's shared memory

Every block has the **same layout**, so every block knows where every other block's areas are:

- a **send area**: all the messages it has to send, back to back;
- a **receive area**: room for all the messages it will receive, back to back;
- both as large as the largest one over all blocks; an mbarrier in front (copy-engine completion).



Messages are in **rotated order**: block b stores its message to b first, then to $b{+}1, b{+}2, \ldots \pmod{p}$; same for its receive area. So at any moment the blocks work on *different* peers instead of all starting with block 0. Offset tables (`out_start`, `in_start`) say where each message starts.

Unbalanced communication: the a2av() kernel

```
__global__ void a2av(Params params) {
  // setup
  for (int round = 0; round < rounds; round++) {
    fill_send_area(b, round);  // produce
    cluster.sync();       // data ready, areas free
    long long t = clock64();
    exchange(b, cluster, round);  // move bytes
    __syncthreads();
    busy += clock64() - t;
    cluster.sync();       // everything delivered
    check_recv_area(b, round);    // consume
  }
}
```

- one block per rank, one cluster
- **setup**: decide between receiver's receive area (push) or the sender's send area, in DSMEM (pull) or in global memory (gmem)
- exchange() picks the method (next slide)
- all threads copy the area **word by word, 16 B each**
- busy: this block's time until it is done; the exchange time is barrier to barrier

Unbalanced communication: five ways to move the bytes

Directly SM to SM, through DSMEM

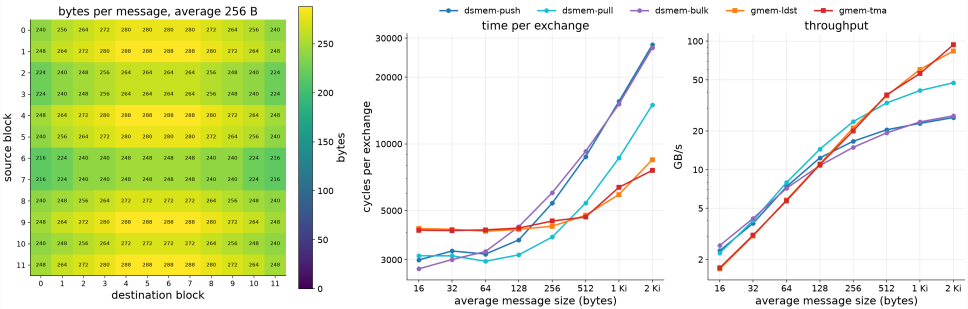
- **dsmem-push**: my threads copy my *send area* into the receivers' *receive areas*.
- **dsmem-pull**: my threads fill my *receive area* by reading the senders' *send areas*.
- **dsmem-bulk**: thread k asks the copy engine (TMA) to copy my message k into its receiver; the receiver's mbarrier counts the bytes landed.

Through global memory

- **gmem-ldst**: every block copies its whole send area to its slot in global memory, `cluster.sync()`, then gathers its incoming messages from the senders' slots.
- **gmem-tma**: the same two copies by the copy engine: one copy out per block, one copy in per message.

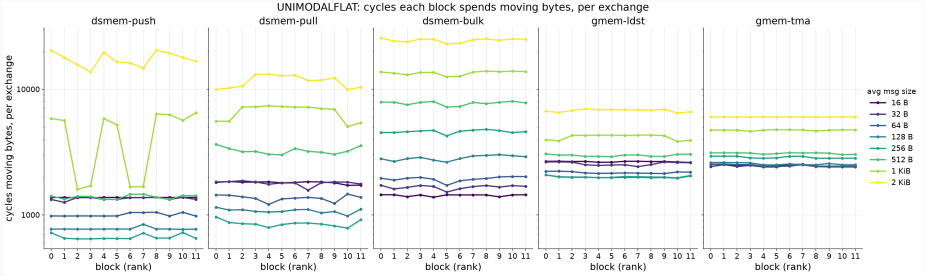
unimodalflat: the balanced reference

- Almost uniform ($\propto 1 + 0.2\,(1 - x^2)$): every block sends 0.91–1.06× the mean, every message is close to the average (216–288 B when the average is 256 B).
- Up to 2 KiB: at 4 KiB the send and receive areas no longer fit in shared memory.
- DSMEM is faster up to 256 B (fewer barriers: 2 against 3), global memory from 512 B; at 2 KiB gmem-tma takes 7.6 k cycles, dsmem-pull 15 k, push and bulk 28 k.



unimodalflat: per block

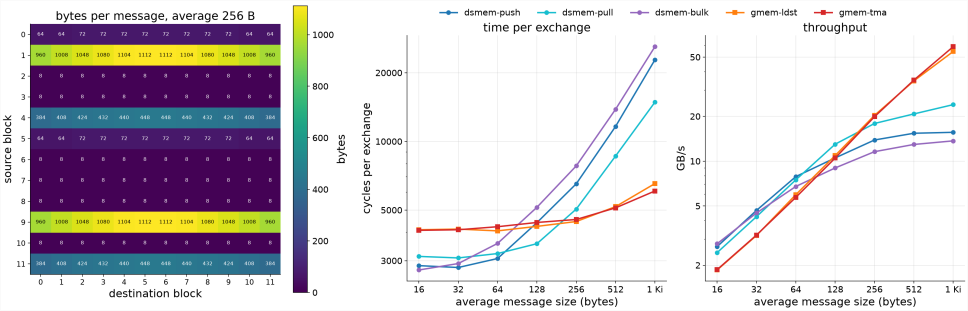
Busy cycles of each block per exchange: from the first `cluster.sync()` until the block is done (bulk: until all its incoming bytes have landed; gmem: the middle `cluster.sync()` included).



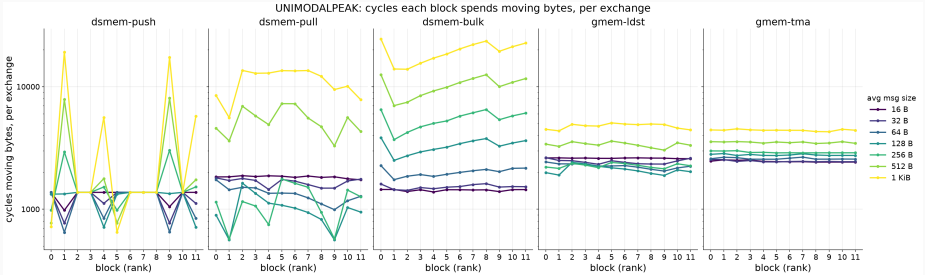
- Balanced pattern, balanced blocks: no block stands out, for any method.
- Not explained yet: dsmem-push at 1 KiB, blocks 2, 3, 6, 7 (the lightest senders, -4 to -9%) finish 3–4$\times$ earlier.

unimodalpeak: two heavy senders

- A Gaussian cut into 12 equal slices: blocks 1 and 9 send 4× the mean (67% of all bytes), blocks 4 and 11 1.6×, the other 8 almost nothing. Receives stay balanced ($\leq 1.06\times$ the mean).
- Up to 1 KiB: at 2 KiB the heaviest send area alone is 98 KiB.
- DSMEM is faster up to 128 B, global memory from 256 B (one step earlier than UNIMODALFLAT).



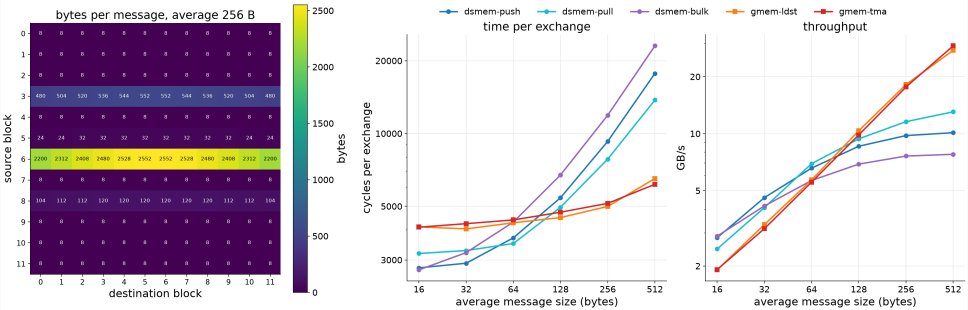
unimodalpeak: per block



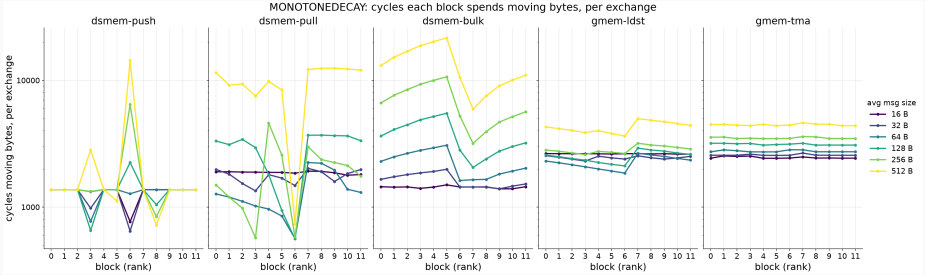
- **push**: spikes at the heavy senders 1, 9 (and 4, 11): the exchange waits for them.
- **pull**: most incoming bytes sit in blocks 1 and 9, so they read them locally and are the fastest.
- **bulk**: each sender queues its copies in rotated order $(b, b{+}1, \dots)$: the blocks right after a heavy sender get their bytes first, those just before it last (ramp from 1 to 8, from 9 to 0).
- **gmem**: flat, the middle `cluster.sync()` makes every block wait for the slowest.

monotonedecay: one dominant sender

- e^{-x} cut into 12 equal slices: block 6 sends 9× the mean (77% of all bytes), block 3 2×, the other 10 almost nothing. Receives stay balanced.
- Only up to 512 B: at 1 KiB block 6's send area is 113 KiB.
- DSMEM is faster only up to 64 B. From UNIMODALFLAT to here, at 512 B, dsmem-pull gets 2.5× slower (5.4 k → 13.8 k cycles), gmem-tma only 1.3× (4.7 k → 6.2 k).



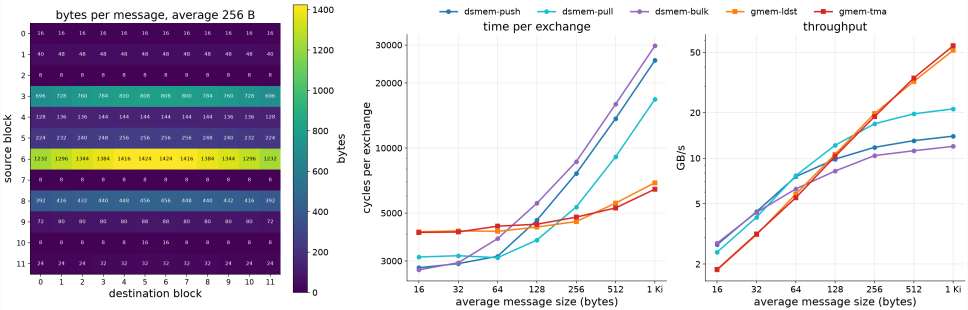
monotonedecay: per block



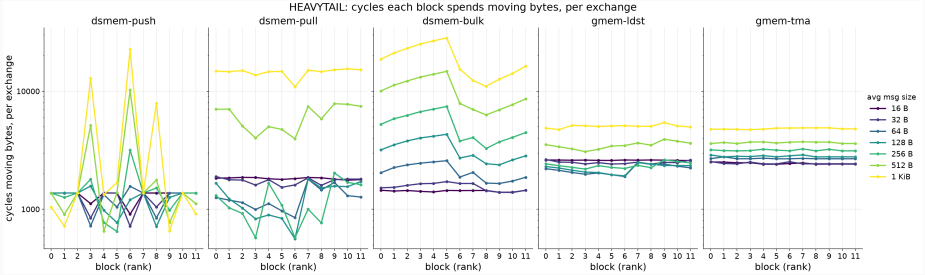
- **push**: block 6 alone writes 77% of the bytes: 14 k cycles at 512 B, the others ∼1.4 k.
- **pull**: the other 11 blocks all read from SM 6's shared memory; block 6 reads locally and is the fastest.
- **bulk**: one ramp starting at block 6 (6, 7, . . . , 11, 0, . . . , 5): the order in which block 6's copies are queued.
- **gmem**: flat again.

heavytail: a few heavy senders

- Pareto ($\alpha = 2.5$) cut into 12 log-spaced slices: block 6 sends 5× the mean (43% of all bytes), blocks 3, 8 and 5 the next largest, then a long tail. Receives stay balanced.
- Up to 1 KiB: at 2 KiB block 6's send area is 126 KiB.
- DSMEM is faster up to 128 B, global memory from 256 B; at 1 KiB gmem-tma takes 6.5 k cycles, dsmem-pull 17 k, push 25 k, bulk 30 k.



heavytail: per block



- **push**: one spike per heavy sender, in order of size: 6, 3, 8.
- **pull**: dips at 6 (and 3) for the same reason as before: they read their own bytes locally.
- **bulk**: the ramp starts at block 6 again.
- **gmem**: flat. The imbalance is on the *send* side: with DSMEM the heavy sender's SM is the bottleneck (it writes, or is read by, everyone); with global memory it only copies its area out once and the reads come from L2.